

Prolog - Basics

Multi-Agent Systems
Practical 1

Prolog Syntax (1)

- :- defines an implication ($A \rightarrow B$ corresponds to $B :- A$).
- , defines a conjunction (in right-hand side of rule)
- ; defines a disjunction (but can also be multiple rules)

An **atom** is one of the following:

1. A sequence of letters, digits, or underscores that *starts with a lower-case letter*, e.g. car, hOUSE_2, father.
2. A sequence of letters, digits or underscores that are *enclosed in single quotes*, e.g. 'car', 'House', 'HOUSE 2'.
3. A string of special characters, e.g. :- or ,

A **number** is any sequence of numbers, possibly containing a dot.

Both atoms and numbers are called **constants**.

Prolog Syntax (2)

A **variable** is a sequence of letters, digits or underscores that *starts with an upper-case letter or an underscore*, e.g. X, Car, HOUSE_2, _father, House.

- The variable _ is called the **anonymous variable**.

A **term** is defined recursively:

1. Each constant and variable is a term.
2. $f(t_1, \dots, t_k)$ is a term if each t_i is a term and f is an atom.
 - This is called a **compound** term with **functor** f and **arity** k , which is often referred to in the notation f/k (**signature**), e.g. has_wheels/1 or has_embodiment/2.
 - A term is **ground** when it contains no variables.

Prolog Syntax (3)

Any constant or compound term is known as a **predicate**.

If immediately followed by a dot (.), it is called a **fact** (knowledge).

Otherwise, it can be a **rule** if:

- Given by $A_0 :- A_1, \dots, A_n$. with $n \geq 0$ and each A_i a predicate.

A_0 is called the **head** of the rule, $A_1, \dots, A_n$ the **body**.

A **clause** is any fact or rule.

- $p(X) :- f(X), r(X)$ (Prolog)
- $f(X) \wedge r(X) \rightarrow p(X)$ (FOL)

Clauses are always **universal**, i.e. here $p(X)$ holds for all X .

Prolog Syntax (4)

- A **Prolog program** is a finite sequence of clauses.
- The set of all facts and rules that use a certain predicate P is called the **definition** of that predicate. If there is none in a certain program, P is called **undefined** (cannot be queried).
- A **built-in** predicate is defined in SWI Prolog itself, e.g. fail/0, true/0, consult/1, member/2, nth0/3, etcetera.
- A **query** is defined as ‘?- p.’, where p is some predicate, known then as a **goal**.
 - Queries are existential, i.e., ?- p(X) corresponds to $\exists X p(X)$ in FOL.

Querying

Knowledge base

```
nao(hero).  
aibo(daisy).  
aibo(cooper).  
pepper(ada).  
ugv(charlie).
```

```
has_wheels(R) :- pepper(R);ugv(R).  
social(R) :- nao(R);aibo(R);pepper(R).
```

```
has_embodiment(R, humanoid) :- nao(R);pepper(R).  
has_embodiment(R, zoomorphic) :- aibo(R).  
has_embodiment(R, machinelike) :- ugv(R).
```

Queries

1. Derive your own answers for the following queries first.
2. Open SWI-Prolog and see for your self.

```
?- nao(hero).
```

```
?- nao(Hero).
```

```
?- Nao(hero).
```

```
?- has_wheels(R), social(R).
```

```
?- has_wheels(R); social(R). (Why is ada mentioned twice?)
```

```
?- has_embodiment(R, humanoid), social(R).
```

```
?- has_embodiment(R, humanoid), \+ social(R).
```

Building a knowledge base

Create a knowledge base from the following information

Story:

In a small library, several books sit together on a shelf. Each book belongs to a specific genre, has an author, and is either available or checked out. A librarian, Sarah, manages the library, while patrons like Alice and Bob visit to borrow books.

- Sarah is the librarian.
- Alice and Bob are patrons.
- Three books are available on the shelf: *The Great Gatsby*, *1984*, and *To Kill a Mockingbird*.
 - *The Great Gatsby* is a fiction book by F. Scott Fitzgerald and is currently available.
 - *1984* is a dystopian novel by George Orwell, and Alice has borrowed it.
 - *To Kill a Mockingbird* is a fiction book by Harper Lee, and it is available.

Relationships:

- Sarah manages the library.
- Alice and Bob are patrons who can borrow books.
- Each book has a title, author, genre, and availability status.
- Patrons can borrow or return books.

Solution

```
% Facts
```

```
% Define librarian and patrons
```

```
librarian(sarah).
```

```
patron(alice).
```

```
patron(bob).
```

```
% Define books with their title, author, genre, and  
availability
```

```
book('The Great Gatsby', 'F. Scott Fitzgerald', fiction,  
available).
```

```
book('1984', 'George Orwell', dystopian, borrowed).
```

```
book('To Kill a Mockingbird', 'Harper Lee', fiction,  
available).
```

```
% Define borrowed relationships (who has borrowed what)
```

```
borrowed(alice, '1984').
```

```
% Define who manages the library
```

```
manages(sarah, library).
```

```
% Rules
```

```
% Check if a person can borrow a book: A person can borrow a book if they are
```

```
% a patron, the book is available, and they haven't already borrowed it.
```

```
can_borrow(Person, Title) :-
```

```
    patron(Person),
```

```
    book(Title, _, _, available),
```

```
    \+ borrowed(Person, Title).
```

```
% Change a book's status to borrowed when a patron borrows it
```

```
borrow_book(Person, Title) :-
```

```
    can_borrow(Person, Title),
```

```
    retract(book(Title, Author, Genre, available)),
```

```
    assert(book(Title, Author, Genre, borrowed)),
```

```
    assert(borrowed(Person, Title)).
```

```
% Return a borrowed book, changing its status back to available
```

```
return_book(Person, Title) :-
```

```
    borrowed(Person, Title),
```

```
    retract(book(Title, Author, Genre, borrowed)),
```

```
    assert(book(Title, Author, Genre, available)),
```

```
    retract(borrowed(Person, Title)).
```

```
% Find books by genre
```

```
find_books_by_genre(Genre, Title) :-
```

```
    book(Title, _, Genre, _).
```